PROJET SYSTÈME ET RÉSEAUX (ETE7-PROJ1) : CRÉATION D'UN SYSTÈME DE MÉMOIRE PARTAGÉE DISTRIBUÉE (DSM)

Fadwa Abakarim
Guillaume Mercier
Philippe Swartvagher

Département Télécommunications — ENSEIRB-MMK — Bordeaux-INP

2026—2027

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau
- La production de code est un objectif secondaire
- L'objectif principal est d'évaluer votre niveau d'expertise

Vous devez être capables :

- Évolution : examen pendant la session de Janvier

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau

⇒ La **production de code** est un objectif **secondaire**

⇒ L'objectif principal est d'évaluer votre **niveau d'expertise**

Vous devez être capables :

- de justifier tous vos choix techniques
- d'analyser de façon critique votre production (qu'est ce qui est améliorable ? etc.)

- Évaluation : examen pendant la session de Janvier

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau

⇒ La **production de code** est un objectif **secondaire**

⇒ L'objectif principal est d'évaluer votre **niveau d'expertise**

Vous devez être capables :

- de justifier tous vos choix techniques
- d'analyser de façon critique votre production (qu'est ce qui est améliorable ? etc.)

- Évaluation : examen pendant la session de Janvier

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau

⇒ La **production de code** est un objectif **secondaire**

⇒ L'objectif principal est d'évaluer votre **niveau d'expertise**

Vous devez être capables :

- de justifier tous vos choix techniques
- d'analyser de façon critique votre production (qu'est ce qui est améliorable ? etc.)

- Évaluation : examen pendant la session de Janvier

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau

⇒ La **production de code** est un objectif **secondaire**

⇒ L'objectif principal est d'évaluer votre **niveau d'expertise**

Vous devez être capables :

- de justifier tous vos choix techniques
- d'analyser de façon critique votre production (qu'est ce qui est améliorable ? etc.)

- Évaluation : examen pendant la session de Janvier

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau

⇒ La **production de code** est un objectif **secondaire**

⇒ L'objectif principal est d'évaluer votre **niveau d'expertise**

Vous devez être capables :

- de justifier tous vos choix techniques
- d'analyser de façon critique votre production (qu'est ce qui est améliorable ?, etc.)

- Évaluation : examen pendant la session de Janvier

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau

⇒ La **production de code** est un objectif **secondaire**

⇒ L'objectif principal est d'évaluer votre **niveau d'expertise**

Vous devez être capables :

— de justifier tous vos choix techniques
— d'analyser de façon critique votre production (qu'est ce qui est améliorable ?, etc.)

- Évaluation : examen pendant la session de Janvier

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau

⇒ La **production de code** est un objectif **secondaire**

⇒ L'objectif principal est d'évaluer votre **niveau d'expertise**

Vous devez être capables :

— de justifier tous vos choix techniques
— d'analyser de façon critique votre production (qu'est ce qui est améliorable ?, etc.)

- Évaluation : examen pendant la session de Janvier

- Consolidation des compétences en programmation impérative C
- Application et consolidation des connaissances acquises en programmation système
- Application et consolidation des connaissances acquises en programmation réseau

⇒ La **production de code** est un objectif **secondaire**

⇒ L'objectif principal est d'évaluer votre **niveau d'expertise**

Vous devez être capables :

— de justifier tous vos choix techniques
— d'analyser de façon critique votre production (qu'est ce qui est améliorable ?, etc.)

- Évaluation : examen pendant la session de Janvier

- Il est normal de ne pas tout comprendre du premier coup !
- Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques
- Merci de lire attentivement le sujet avant de débuter le travail

- Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail individuel en monôme
- Nous encourageons les échanges d'idées
- Il est préférable de discuter entre vous qu'avec une IA

- Il est normal de ne pas tout comprendre du premier coup !

⇒ Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques

⇒ Merci de lire attentivement le sujet avant de débuter le travail

⇒ L'analyse doit précéder le développement

⇒ Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail **individuel** en monôme

⇒ Nous encourageons les échanges d'idées

⇒ Il est préférable de discuter entre vous qu'avec une IA

- Il est normal de ne pas tout comprendre du premier coup !

⇒ Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques

⇒ Merci de lire attentivement le sujet avant de débuter le travail

⇒ ... l'analyse doit précéder le développement

⇒ Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail **individuel** en monôme

⇒ Nous encourageons les échanges d'idées

⇒ Il est préférable de discuter entre vous qu'avec une IA

- Il est normal de ne pas tout comprendre du premier coup !

⇒ Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques

⇒ Merci de lire attentivement le sujet avant de débuter le travail

- l'analyse doit précéder le développement

⇒ Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail **individuel** en monôme

⇒ Nous encourageons les échanges d'idées

⇒ Il est préférable de discuter entre vous qu'avec une IA

- Il est normal de ne pas tout comprendre du premier coup !

⇒ Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques

⇒ Merci de lire attentivement le sujet avant de débuter le travail

- l'analyse doit précéder le développement

⇒ Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail **individuel** en monôme

⇒ Nous encourageons les échanges d'idées

⇒ Il est préférable de discuter entre vous qu'avec une IA

- Il est normal de ne pas tout comprendre du premier coup !

⇒ Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques

⇒ Merci de lire attentivement le sujet avant de débuter le travail

- l'analyse doit précéder le développement

⇒ Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail **individuel** en monôme

⇒ Nous encourageons les échanges d'idées

⇒ Il est préférable de discuter entre vous qu'avec une IA

- Il est normal de ne pas tout comprendre du premier coup !

⇒ Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques

⇒ Merci de lire attentivement le sujet avant de débuter le travail

- l'analyse doit précéder le développement

⇒ Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail **individuel** en monôme

⇒ Nous encourageons les échanges d'idées

⇒ Il est préférable de discuter entre vous qu'avec une IA

- Il est normal de ne pas tout comprendre du premier coup !

⇒ Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques

⇒ Merci de lire attentivement le sujet avant de débuter le travail

- l'analyse doit précéder le développement

⇒ Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail **individuel** en monôme

⇒ Nous encourageons les échanges d'idées

⇒ Il est préférable de discuter entre vous qu'avec une IA

- Il est normal de ne pas tout comprendre du premier coup !

⇒ Les séances de projet seront l'occasion de revenir sur le sujet et d'éclaircir les points (potentiellement) problématiques

⇒ Merci de lire attentivement le sujet avant de débuter le travail

  - l'analyse doit précéder le développement

⇒ Le rôle des encadrants est de vous accompagner : n'hésitez pas à les solliciter

- Travail **individuel** en monôme

⇒ Nous encourageons les échanges d'idées

⇒ Il est préférable de discuter entre vous qu'avec une IA

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes
  - Nécessité de **déployer** l'application
    - Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique
    - Utilisation d'un **outil annexe** dédié
  - Nécessité de gérer les **communications** inter-processus
    - Plusieurs modes de communication possibles

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications explicites : l'application utilise directement l'API socket (par exemple)
  - Communications implicites : l'application utilise une bibliothèque qui masque l'API réseau
  - Cas des systèmes basés sur de la mémoire partagée distribuée

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications explicites : l'application utilise directement l'API socket (par exemple)
  - Communications implicites : l'application utilise une bibliothèque qui masque l'API réseau
  - Cas des systèmes basés sur de la mémoire partagée distribuée

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications explicites : l'application utilise directement l'API socket (par exemple)
  - Communications implicites : l'application utilise une bibliothèque qui masque l'API réseau
  - Cas des systèmes basés sur de la mémoire partagée distribuée

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications explicites : l'application utilise directement l'API socket (par exemple)
  - Communications implicites : l'application utilise une bibliothèque qui masque l'API réseau
  - Cas des systèmes basés sur de la mémoire partagée distribuée

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications explicites : l'application utilise directement l'API socket (par exemple)
  - Communications implicites : l'application utilise une bibliothèque qui masque l'API réseau
  - Cas des systèmes basés sur de la mémoire partagée distribuée

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications explicites : l'application utilise directement l'API socket (par exemple)
  - Communications implicites : l'application utilise une bibliothèque qui masque l'API réseau
  - Cas des systèmes basés sur de la mémoire partagée distribuée

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications explicites : l'application utilise directement l'API socket (par exemple)
  - Communications implicites : l'application utilise une bibliothèque qui masque l'API réseau
  - Cas des systèmes basés sur de la mémoire partagée distribuée

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications explicites : l'application utilise directement l'API socket (par exemple)
  - Communications implicites : l'application utilise une bibliothèque qui masque l'API réseau
  - Cas des systèmes basés sur de la mémoire partagée distribuée

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications **explicites** : l'application utilise directement l'API socket (par exemple)
  - Communications **implicites** : l'application utilise une **bibliothèque** qui masque l'API réseau
  - ⇒ Cas des systèmes basés sur de la *mémoire partagée distribuée*

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications **explicites** : l'application utilise directement l'API socket (par exemple)
  - Communications **implicites** : l'application utilise une **bibliothèque** qui masque l'API réseau
  - ⇒ Cas des systèmes basés sur de la *mémoire partagée distribuée*

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications **explicites** : l'application utilise directement l'API socket (par exemple)
  - Communications **implicites** : l'application utilise une **bibliothèque** qui masque l'API réseau
    ⇒ Cas des systèmes basés sur de la *mémoire partagée distribuée*

Thématique du projet

Cadre général : applications **réparties/distribuées**

- Processus s'exécutant sur des machines physiquement distinctes

- Nécessité de **déployer** l'application

⇒ Déploiement manuel théoriquement possible, mais pratiquement trop difficile/peu pratique

⇒ Utilisation d'un **outil annexe dédié**

- Nécessité de gérer les **communications** inter-processus
  - Échanges de données
  - Synchronisations
  - etc.

- Plusieurs modes de communication possibles
  - Communications **explicites** : l'application utilise directement l'API socket (par exemple)
  - Communications **implicites** : l'application utilise une **bibliothèque** qui masque l'API réseau
  - ⇒ Cas des systèmes basés sur de la *mémoire partagée distribuée*

Fonctionnement de la mémoire partagée distribuée

Principe général

- Mémoire partagée distribuée : *Distributed Shared Memory (DSM)*

- Une partie de l'espace d'adressage des processus est partagée
- Une écriture dans cette zone par un processus est lisible par tous les autres processus

- Chaque processus conserve et manipule sa propre mémoire (en dehors de la zone partagée)

- Mémoire partagée distribuée : *Distributed Shared Memory (DSM)*

- Une partie de l'espace d'adressage des processus est partagée

⇒ Une écriture dans cette zone par un processus est lisible par tous les autres processus

- Chaque processus conserve et manipule sa propre mémoire (en-dehors de la zone partagée)

- Mémoire partagée distribuée : *Distributed Shared Memory (DSM)*

- Une partie de l'espace d'adressage des processus est partagée

⇒ Une écriture dans cette zone par un processus est lisible par tous les autres processus

- Chaque processus conserve et manipule sa propre mémoire (en-dehors de la zone partagée)

- Mémoire partagée distribuée : *Distributed Shared Memory (DSM)*

- Une partie de l'espace d'adressage des processus est partagée

⇒ Une écriture dans cette zone par un processus est lisible par tous les autres processus

- Chaque processus conserve et manipule sa propre mémoire (en-dehors de la zone partagée)

- Mémoire partagée distribuée : *Distributed Shared Memory (DSM)*

- Une partie de l'espace d'adressage des processus est partagée

⇒ Une écriture dans cette zone par un processus est lisible par tous les autres processus

- Chaque processus conserve et manipule sa propre mémoire (en-dehors de la zone partagée)

Fonctionnement de la mémoire partagée distribuée

Dans le cadre du projet

- Les processus de l'application se partagent une certaine plage d'adresses en mémoire
- Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés pages-mémoire
- Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire
- Si un processus est le propriétaire d'une page, il y accède normalement
- Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur
- Traitant du signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

- Les processus de l'application se partagent une **certaine plage d'adresses en mémoire**

⇒ Cette plage est identique pour tous les processus

- Cette plage est organisée en morceaux de 4ko, appelés **pages-mémoire**

⇒ Les processus accèdent à ces pages en lecture ou écriture

- À un instant donné, seul un processus est le propriétaire effectif d'une page-mémoire

⇒ Si un processus est le propriétaire d'une page, il y accède normalement

⇒ Sinon, cela provoque une erreur de type *segmentation fault*

- Comme la page est théoriquement accessible, il faut gérer cette erreur

⇒ Traitant de signal adapté

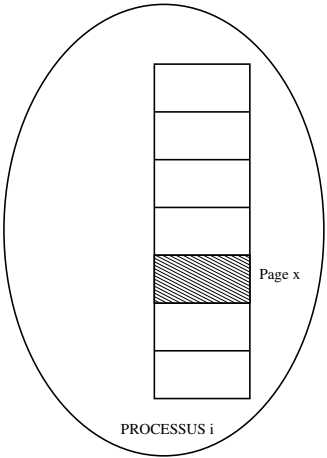
Fonctionnement de la mémoire partagée distribuée

Un exemple

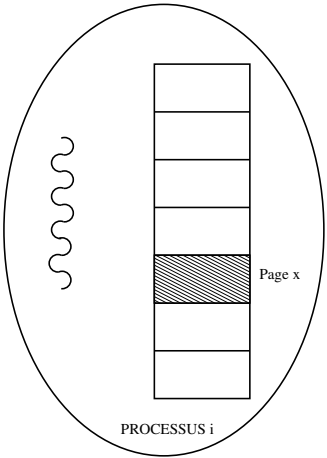
1. Accès propriétaire
2. Accès non propriétaire

1. Accès propriétaire
2. Accès non-propriétaire

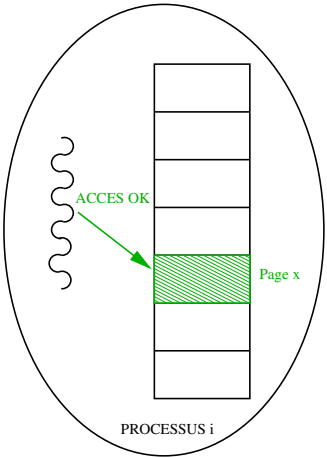
1. Accès propriétaire
2. Accès non-propriétaire



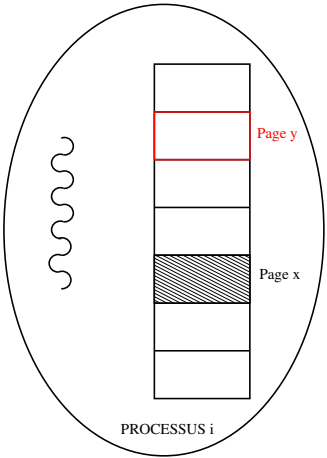
1. Accès propriétaire
2. Accès non-propriétaire



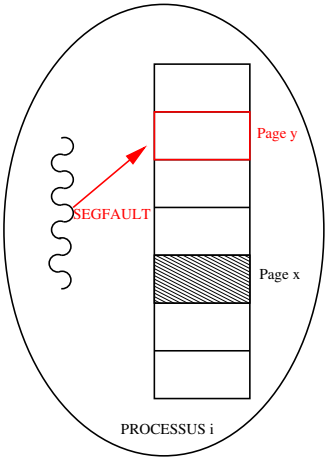
1. Accès propriétaire
2. Accès non-propriétaire



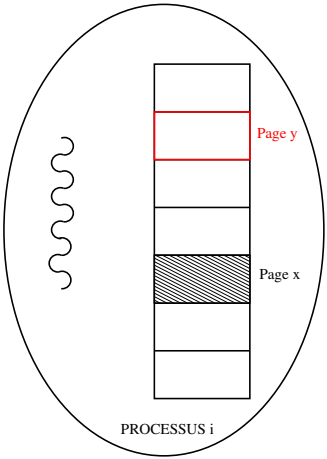
1. Accès propriétaire
2. Accès non-propriétaire

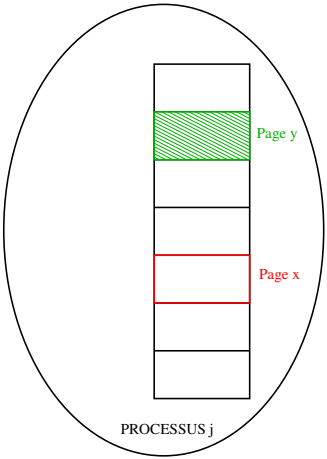


1. Accès propriétaire
2. Accès non-propriétaire

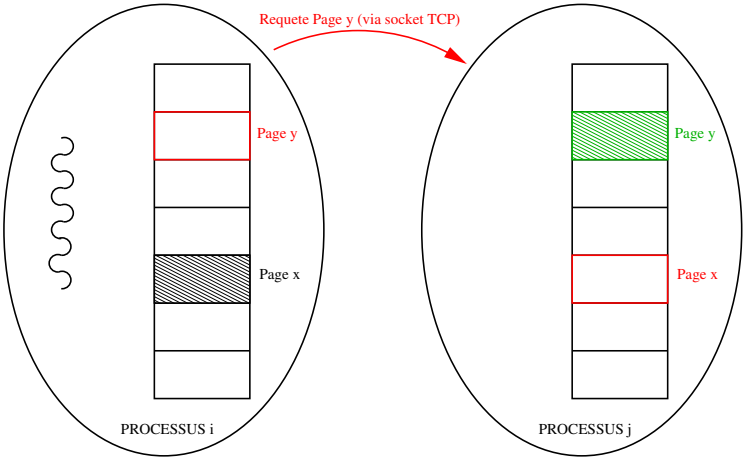


1. Accès propriétaire
2. Accès non-propriétaire

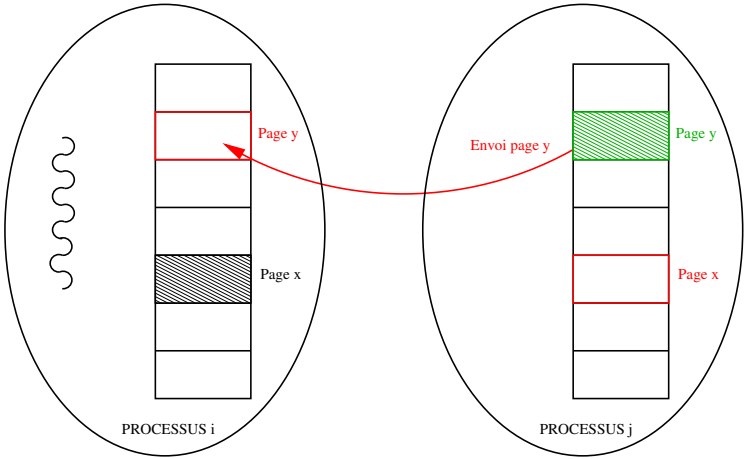




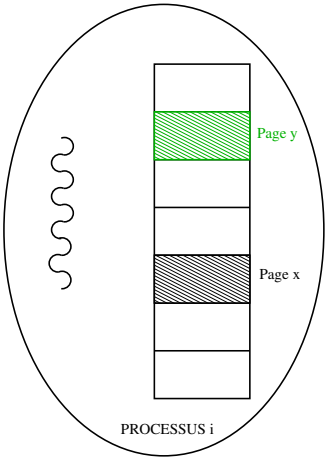
1. Accès propriétaire
2. Accès non-propriétaire

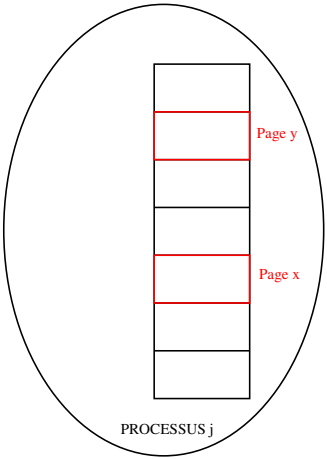


1. Accès propriétaire
2. Accès non-propriétaire

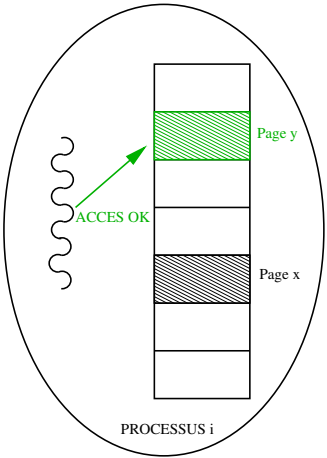


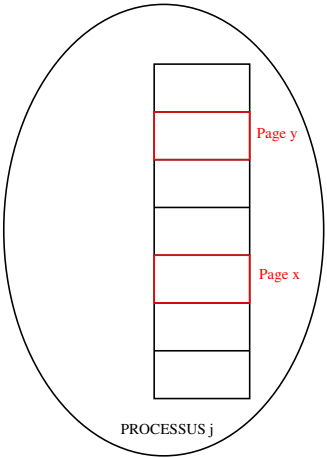
1. Accès propriétaire
2. Accès non-propriétaire





1. Accès propriétaire
2. Accès non-propriétaire





Productions attendues

Deux productions logicielles attendues :

- Production 1 : un programme (daemexec) permettant le déploiement d'une application distribuée
- Production 2 : une bibliothèque (libdsm.a) permettant la gestion des communications au sein des applications distribuées

Productions attendues

Deux productions logicielles attendues :

- Production 1 : un **programme** (`dsmexec`) permettant le déploiement d'une application distribuée
- Production 2 : une **bibliothèque** (`libdsm.a`) permettant la gestion des communications au sein des applications distribuées

Productions attendues

Deux productions logicielles attendues :

- Production 1 : un **programme** (`dsmexec`) permettant le déploiement d'une application distribuée
- Production 2 : une **bibliothèque** (`libdsm.a`) permettant la gestion des communications au sein des applications distribuées

Organisation du travail en phases

Projet organisé en deux phases principales :

1. Phase 1 : Développement du lanceur d'applications basées
2. Phase 2 : Développement de la bibliothèque de mémoire partagée Distex

Chaque phase est décomposée en sous tâches pour :

À la fin de chaque phase (= fin de dernière séance), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - la configuration de votre dépôt Git
    - la création des dépôts privés
    - la définition des variables d'environnement nécessaires pour le travail demandé
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : Lundi 14 Décembre 2026
- Chaque phase est décomposée en sous-tâches pour :
  - faciliter le travail de développement
  - permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - Le logiciel est-il fonctionnel ?
  - Le cahier des charges a-t-il été respecté ?
  - Tests automatiques illustrés pour tous les binômes
  - Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - la configuration de votre dépôt Git
    - la création des dépôts privés
    - la définition des variables d'environnement nécessaires pour le travail demandé
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : Lundi 14 Décembre 2026
- Chaque phase est décomposée en sous-tâches pour :
  - faciliter le travail de développement
  - permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - Le logiciel est il fonctionnel ?
  - La valeur des champs a-t'il été respectés ?
  - Tests automatiques (bashrcus pour tous les binômes)
  - Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : Lundi 14 Décembre 2026
- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - — Tests automatiques (bottiques pour tous les binômes)
  - — Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : Lundi 14 Décembre 2026
- Chaque phase est décomposée en sous-tâches pour :
  - ⇒ faciliter le travail de développement
  - ⇒ permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - ⇒ Le logiciel est il fonctionnel ?
  - ⇒ Le cahier des charges a-t'il été respecté ?
  - ⇒ Tests automatiques (bottiques pour tous les binômes)
  - ⇒ Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : Lundi 14 Décembre 2026
- Chaque phase est décomposée en sous-tâches pour :
  - ⇒ faciliter le travail de développement
  - ⇒ permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - ⇒ Le logiciel est il fonctionnel ?
  - ⇒ Le cahier des charges a-t'il été respecté ?
  - ⇒ Tests automatiques (bottiques pour tous les binômes)
  - ⇒ Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : Lundi 14 Décembre 2026
- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - — Tests automatiques (bottiques pour tous les binômes)
  - — Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : Lundi 14 Décembre 2026
- Chaque phase est décomposée en sous-tâches pour :
  - ⇒ faciliter le travail de développement
  - ⇒ permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - ⇒ Le logiciel est il fonctionnel ?
  - ⇒ La valeur des champs a-t'il été respectés ?
  - ⇒ Tests automatisés (bash/pytest pour tous les binômes)
  - ⇒ Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**
- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - — Tests automatiques (bash/pipes pour tous les binômes)
  - — Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**
- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**
- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail
- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - — Tests automatiques (bash/pipes pour tous les binômes)
  - — Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - — Tests automatiques (batteries pour tous les binômes)
  - — Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - — Tests automatiques (batteries pour tous les binômes)
  - — Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - — Tests automatiques (bash) pour tous les binômes
  - — Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est-il fonctionnel ?
  - — Le cahier des charges a-t-il été respecté ?
  - — Tests automatiques (bottiques pour tous les binômes)
  - — Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est-il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - ⇒ Tests automatisés identiques pour tous les binômes
  - ⇒ Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est-il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - ⇒ Tests automatisés identiques pour tous les binômes
  - ⇒ Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est-il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - ⇒ Tests automatisés identiques pour tous les binômes
  - ⇒ Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est-il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - ⇒ Tests automatisés identiques pour tous les binômes
  - ⇒ Rapport d'exécution des tests disponible dans votre dépôt Git

Organisation du travail en phases

Projet organisé en deux phases principales :

- Phase 1 : Développement du **lanceur d'applications** `dsmexec`
  - ⇒ 11 séances de 1h20 → 14h40 au total
  - ⇒ 1ère séance consacrée à la prise en main (= lecture attentive) du sujet et à :
    - — la configuration de votre dépôt Git
    - — la création des clefs ssh
    - — la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Fin de première phase : **Lundi 23 Novembre 2026**

- Phase 2 : Développement de la **bibliothèque de mémoire partagée** `libdsm`
  - ⇒ 9 séances de 1h20 → 12h au total
  - ⇒ Fin de deuxième phase : **Lundi 14 Décembre 2026**

- Chaque phase est décomposée en sous-tâches pour :
  - — faciliter le travail de développement
  - — permettre une meilleure répartition du travail

- À la fin de chaque phase (= **fin de dernière séance**), nous effectuerons des tests d'exécution pour vérifier les fonctionnalités implémentées
  - — Le logiciel est-il fonctionnel ?
  - — Le cahier des charges a-t'il été respecté ?
  - ⇒ Tests automatisés identiques pour tous les binômes
  - ⇒ Rapport d'exécution des tests disponible dans votre dépôt Git

Phase 1 — Programme dsmexec

Rôles et fonctions

- Rôle : déploiement et « monitoring » d'application distribuée
- Syntaxe : dsmexec machinefile commande arg1 ... argn
- Fonctions :

- Rôle : déploiement et « monitoring » d'application distribuée
- Syntaxe : dsmsexec *machinefile commande* $arg_1 \ldots arg_n$

  ex 1 : dsmexec mon_fichier hello_world

  ⇒ lance le programme hello_world (sans arguments) sur les machines listées dans le fichier mon_fichier

  ex 2 : dsmexec machine1bis calcul 1 2 4 5

  ⇒ lance le programme calcul (avec 4 arguments) sur les machines listées dans le fichier machine1bis
- Fonctions :
  1. Lancement des processus sur les différentes machines listées dans le fichier de machines
  2. Centralisation et affichage des informations produites par les processus distants (affichage sur stdout et stderr)
  3. Distribution des informations nécessaires aux interconnexions des processus distants

- Rôle : déploiement et « monitoring » d'application distribuée
- Syntaxe : `dsmsexec` *machinefile commande* $arg_1 \ldots arg_n$

Ex 1 : `dsmexec mon_fichier hello_world`

⇒ lance le programme `hello_world` (sans arguments) sur les machines listées dans le fichier `mon_fichier`

Ex 2 : `dsmexec mesmachines calcul 1 3 4 5`

⇒ lance le programme `calcul` (avec 4 arguments) sur les machines listées dans le fichier `mesmachines`

- Fonctions :
  1. Lancement des processus sur les différentes machines listées dans le fichier de machines
  2. Centralisation et affichage des informations produites par les processus distants (affichage sur stdout et stderr)
  3. Distribution des informations nécessaires aux interconnexions des processus distants

- Rôle : déploiement et « monitoring » d'application distribuée
- Syntaxe : `dsmsexec` *machinefile commande* $arg_1 \ldots arg_n$

Ex 1 : `dsmexec mon_fichier hello_world`

⇒ lance le programme `hello_world` (sans arguments) sur les machines listées dans le fichier `mon_fichier`

Ex 2 : `dsmexec mesmachines calcul 1 3 4 5`

⇒ lance le programme `calcul` (avec 4 arguments) sur les machines listées dans le fichier `mesmachines`

- Fonctions :
  1. Lancement des processus sur les différentes machines listées dans le fichier de machines
  2. Centralisation et affichage des informations produites par les processus distants (affichage sur stdout et stderr)
  3. Distribution des informations nécessaires aux interconnexions des processus distants

- Rôle : déploiement et « monitoring » d'application distribuée

- Syntaxe : `dsmsexec` *machinefile commande* $arg_1 \ldots arg_n$

Ex 1 : `dsmexec mon_fichier hello_world`

⇒ lance le programme `hello_world` (sans arguments) sur les machines listées dans le fichier `mon_fichier`

Ex 2 : `dsmexec mesmachines calcul 1 3 4 5`

⇒ lance le programme `calcul` (avec 4 arguments) sur les machines listées dans le fichier `mesmachines`

- Fonctions :
  1. Lancement des processus sur les différentes machines listées dans le fichier de machines
  2. Centralisation et affichage des informations produites par les processus distants (affichage sur stdout et stderr)
  3. Distribution des informations nécessaires aux interconnexions des processus distants

- Rôle : déploiement et « monitoring » d'application distribuée

- Syntaxe : `dsmsexec` *machinefile commande* $arg_1 \ldots arg_n$

Ex 1 : `dsmexec mon_fichier hello_world`

⇒ lance le programme `hello_world` (sans arguments) sur les machines listées dans le fichier `mon_fichier`

Ex 2 : `dsmexec mesmachines calcul 1 3 4 5`

⇒ lance le programme `calcul` (avec 4 arguments) sur les machines listées dans le fichier `mesmachines`

- Fonctions :
  1. Lancement des processus sur les différentes machines listées dans le fichier de machines
  2. Centralisation et affichage des informations produites par les processus distants (affichage sur `stdout` et `stderr`)
  3. Distribution des informations nécessaires aux interconnexions des processus distants

- Rôle : déploiement et « monitoring » d'application distribuée

- Syntaxe : `dsmsexec` *machinefile commande* $arg_1 \ldots arg_n$

Ex 1 : `dsmexec mon_fichier hello_world`

⇒ lance le programme `hello_world` (sans arguments) sur les machines listées dans le fichier `mon_fichier`

Ex 2 : `dsmexec mesmachines calcul 1 3 4 5`

⇒ lance le programme `calcul` (avec 4 arguments) sur les machines listées dans le fichier `mesmachines`

- Fonctions :
  1. Lancement des processus sur les différentes machines listées dans le fichier de machines
  2. Centralisation et affichage des informations produites par les processus distants (affichage sur stdout et stderr)
  3. Distribution des informations nécessaires aux interconnexions des processus distants

- Rôle : déploiement et « monitoring » d'application distribuée

- Syntaxe : `dsmsexec` *machinefile commande* $arg_1 \ldots arg_n$

Ex 1 : `dsmexec mon_fichier hello_world`

⇒ lance le programme `hello_world` (sans arguments) sur les machines listées dans le fichier `mon_fichier`

Ex 2 : `dsmexec mesmachines calcul 1 3 4 5`

⇒ lance le programme `calcul` (avec 4 arguments) sur les machines listées dans le fichier `mesmachines`

- Fonctions :
  1. Lancement des processus sur les différentes machines listées dans le fichier de machines
  2. Centralisation et affichage des informations produites par les processus distants (affichage sur `stdout` et `stderr`)
  3. Distribution des informations nécessaires aux interconnexions des processus distants

- Rôle : déploiement et « monitoring » d'application distribuée

- Syntaxe : `dsmsexec` *machinefile commande* $arg_1 \ldots arg_n$

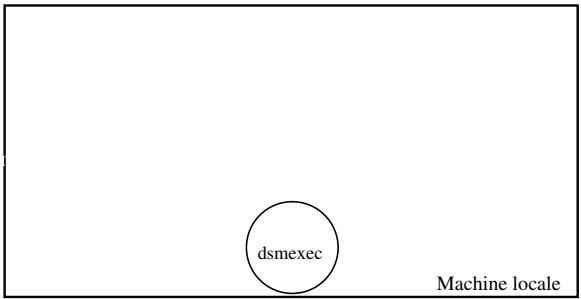
Ex 1 : `dsmexec mon_fichier hello_world`

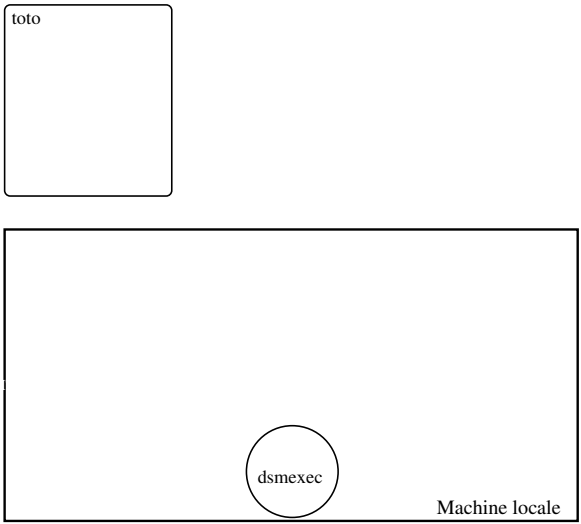
⇒ lance le programme `hello_world` (sans arguments) sur les machines listées dans le fichier `mon_fichier`

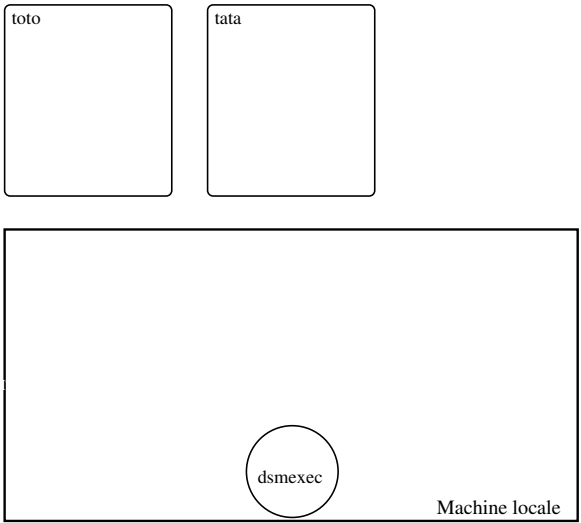
Ex 2 : `dsmexec mesmachines calcul 1 3 4 5`

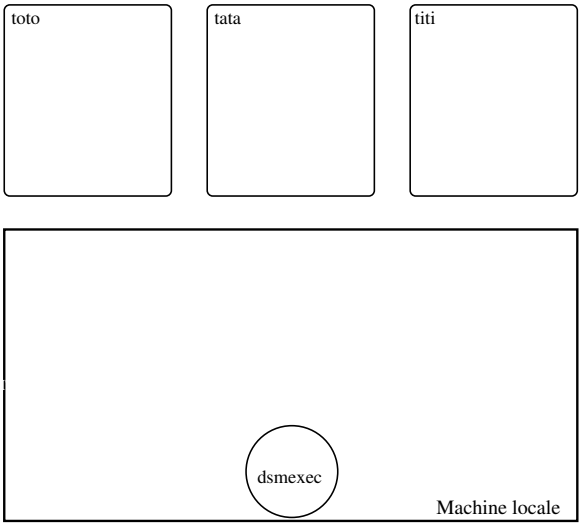
⇒ lance le programme `calcul` (avec 4 arguments) sur les machines listées dans le fichier `mesmachines`

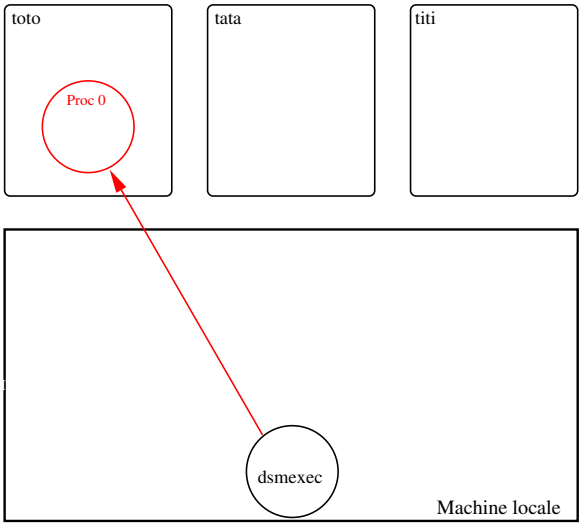
- Fonctions :
  1. Lancement des processus sur les différentes machines listées dans le fichier de machines
  2. Centralisation et affichage des informations produites par les processus distants (affichage sur `stdout` et `stderr`)
  3. Distribution des informations nécessaires aux interconnexions des processus distants

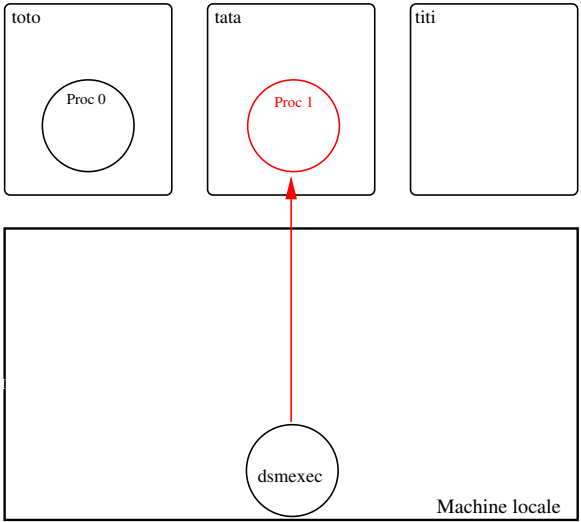

dsmexec
Machine locale

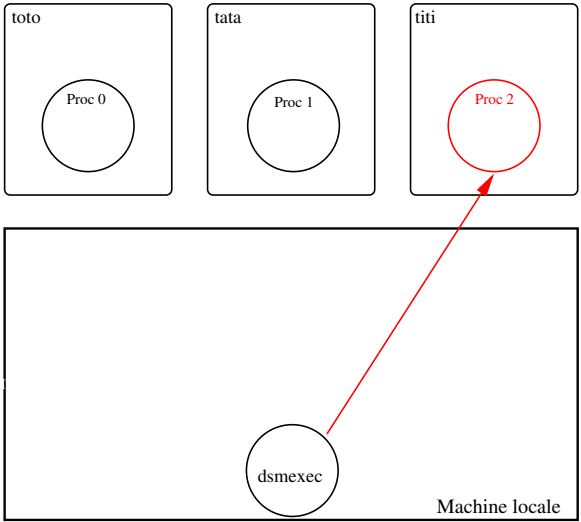

toto
dsmexec
Machine locale

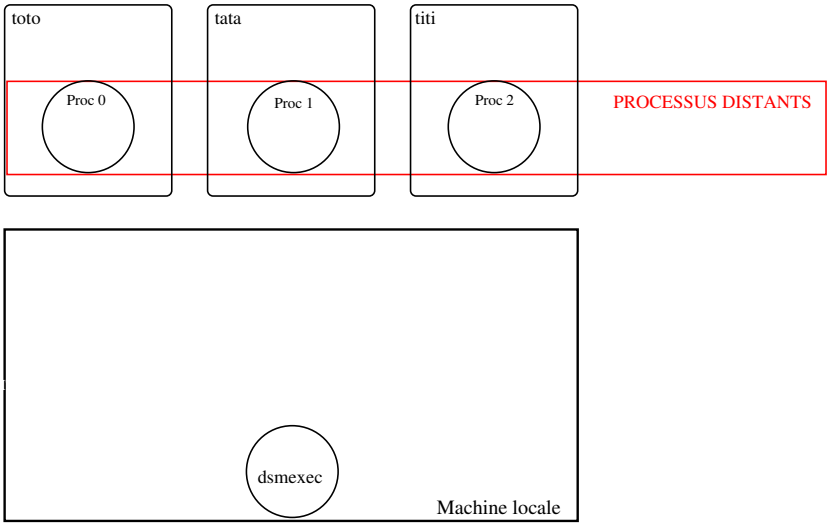

toto
tata
dsmexec
Machine locale

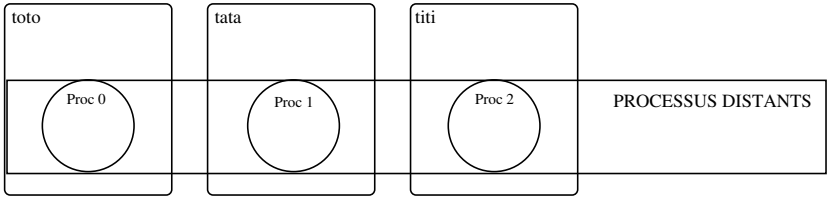

toto
tata
titi
dsmexec
Machine locale

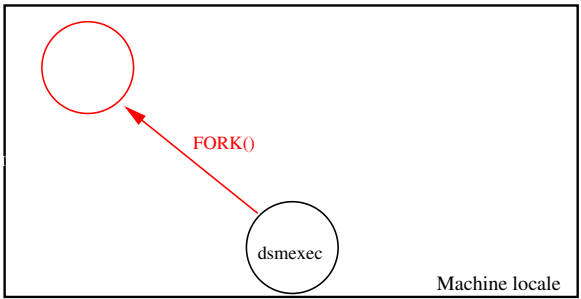

toto
Proc 0
tata
titi
dsmexec
Machine locale

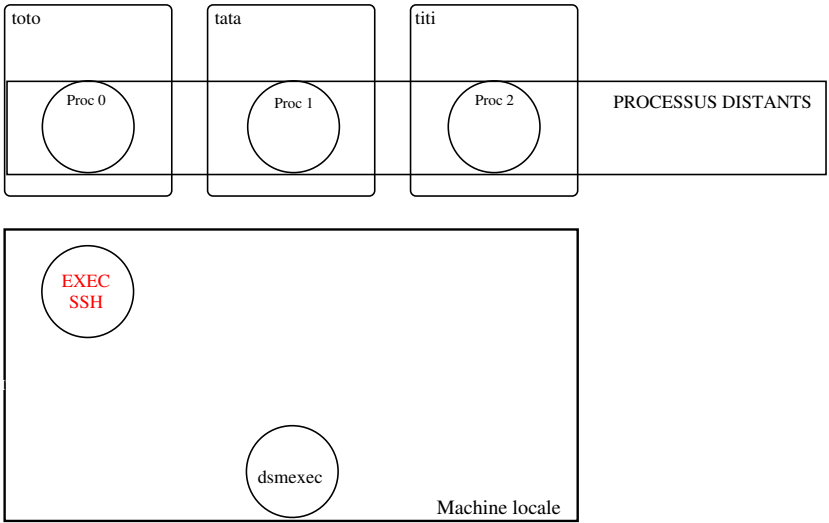

toto
Proc 0
tata
Proc 1
titi
dsmexec
Machine locale

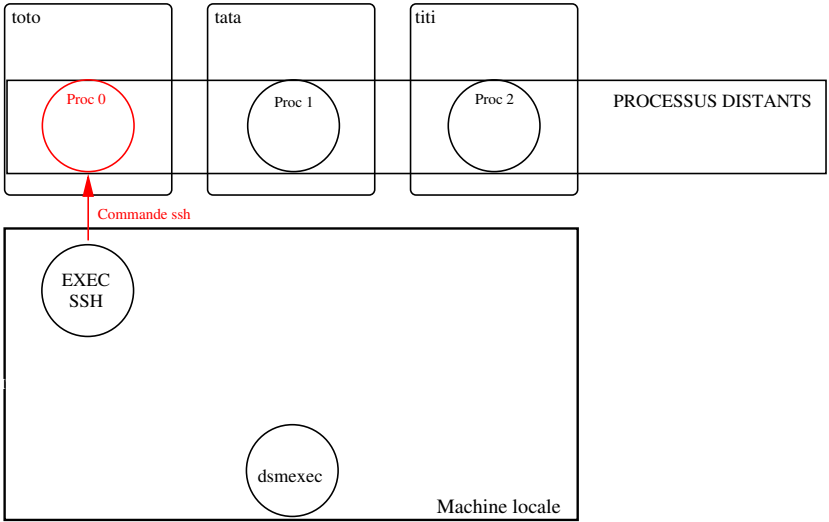

toto
Proc 0
tata
Proc 1
titi
Proc 2
dsmexec
Machine locale

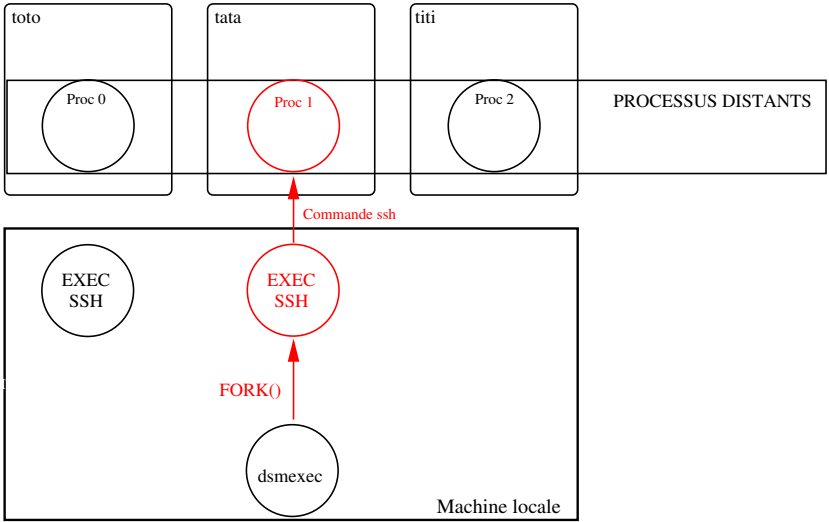

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
dsmexec
Machine locale

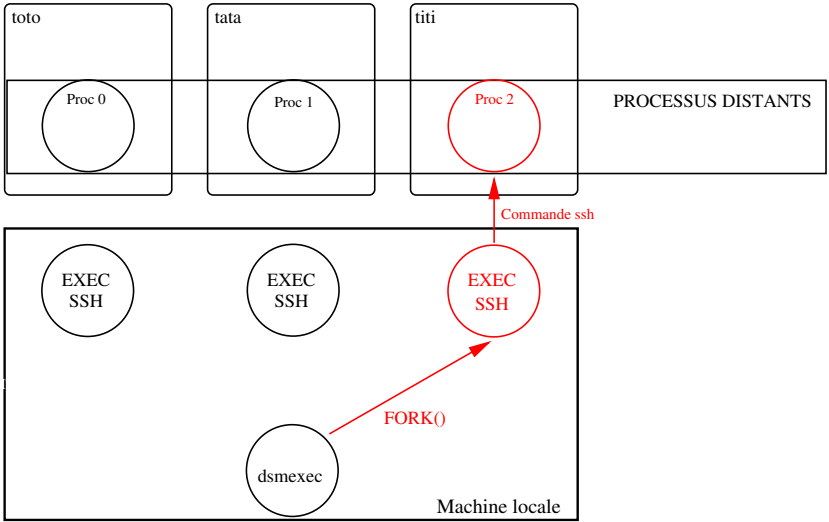

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS

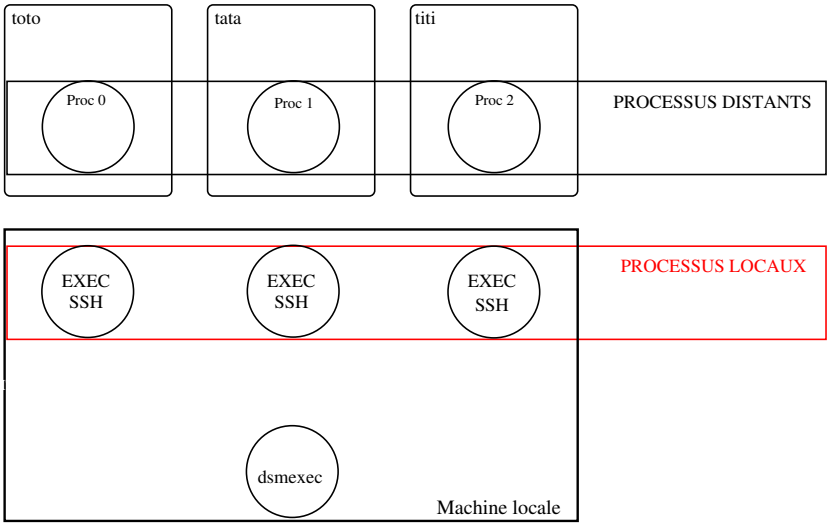

FORK()
dsmexec
Machine locale

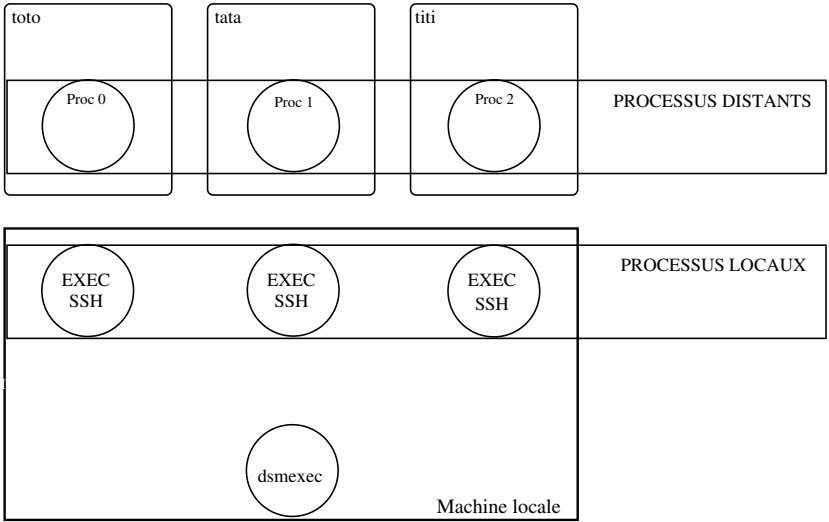

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
EXEC
SSH
dsmexec
Machine locale

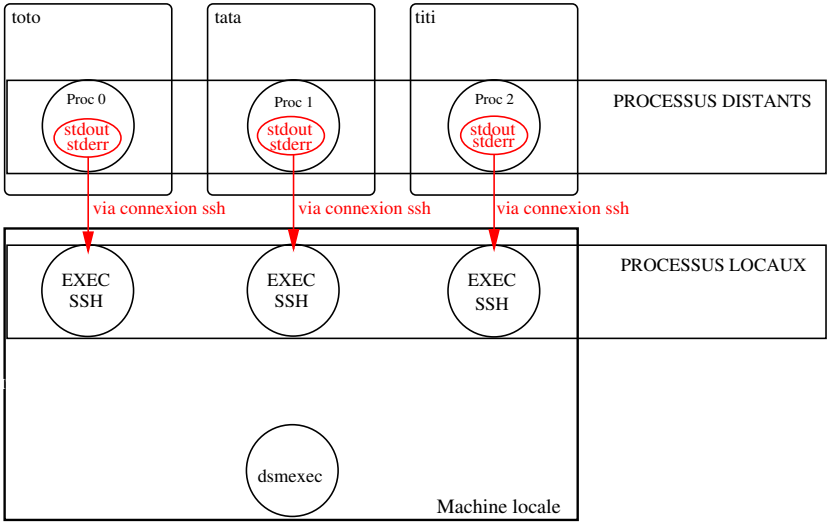

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
Commande ssh
EXEC
SSH
dsmexec
Machine locale

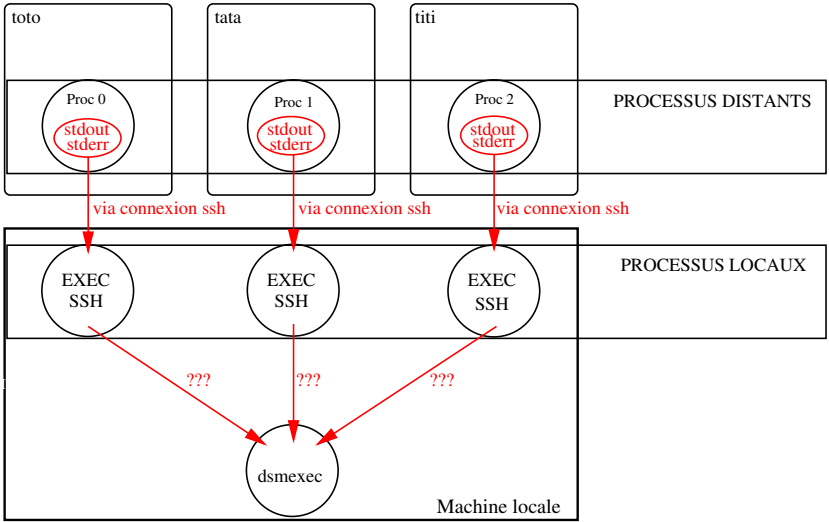

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
Commande ssh
EXEC
SSH
EXEC
SSH
FORK()
dsmexec
Machine locale

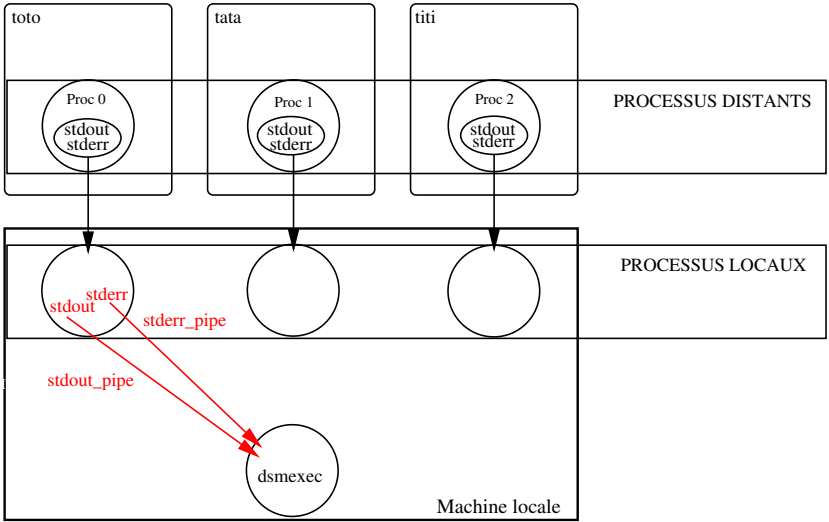

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
Commande ssh
EXEC SSH
EXEC SSH
EXEC SSH
FORK()
dsmexec
Machine locale

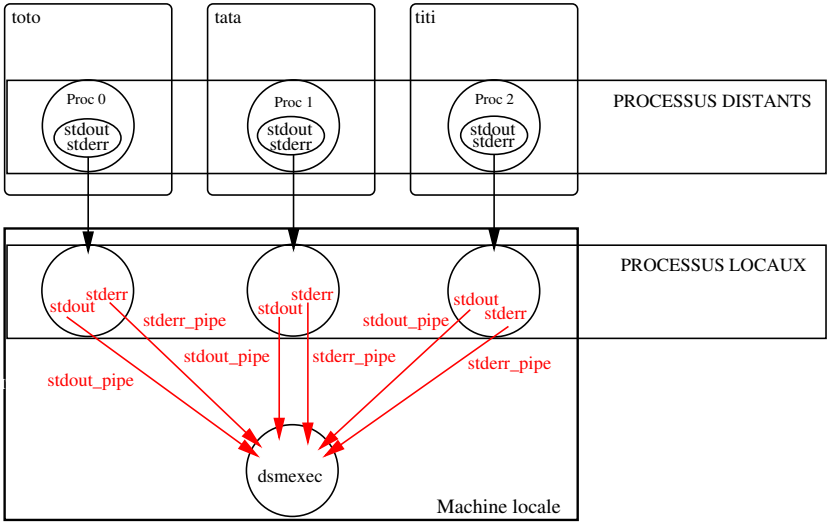

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
EXEC SSH
EXEC SSH
EXEC SSH
PROCESSUS LOCAUX
dsmexec
Machine locale

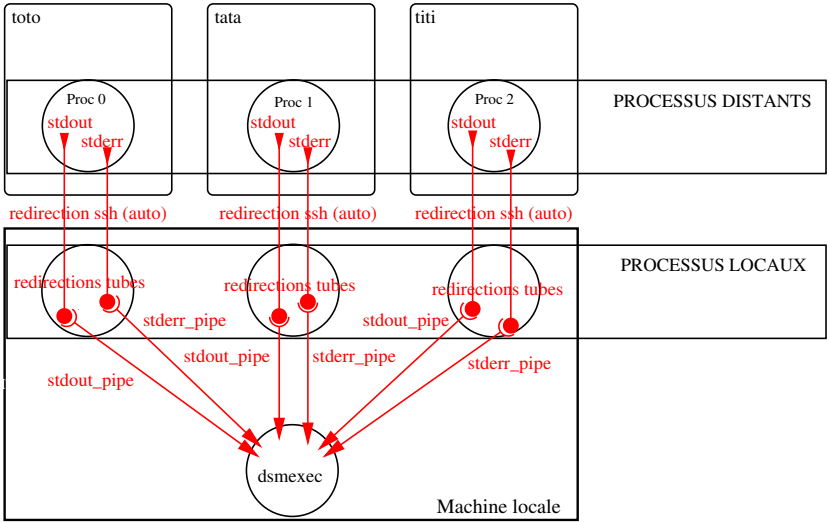

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
EXEC SSH
EXEC SSH
EXEC SSH
PROCESSUS LOCAUX
dsmexec
Machine locale

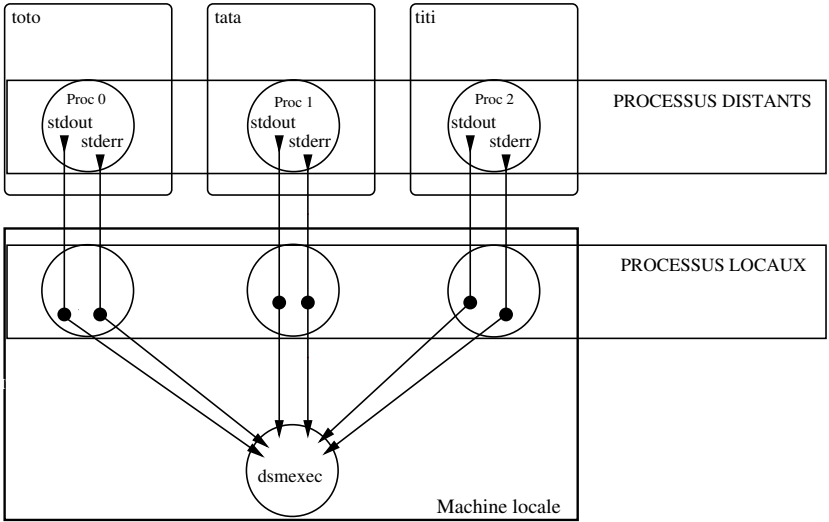

toto
tata
titi
Proc 0
stdout
stderr
Proc 1
stdout
stderr
Proc 2
stdout
stderr
PROCESSUS DISTANTS
via connexion ssh
via connexion ssh
via connexion ssh
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
dsmexec
Machine locale

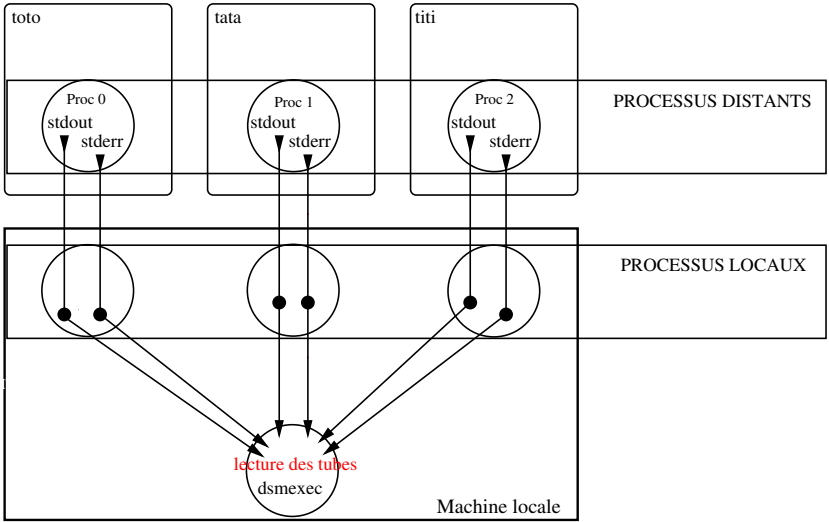

toto
tata
titi
Proc 0
stdout
stderr
Proc 1
stdout
stderr
Proc 2
stdout
stderr
PROCESSUS DISTANTS
via connexion ssh
via connexion ssh
via connexion ssh
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
???
???
???
dsmexec
Machine locale

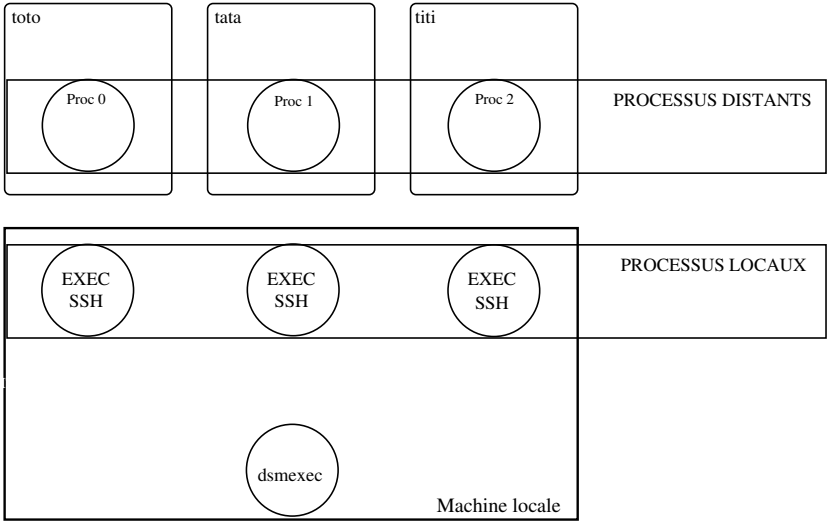

toto
tata
titi
Proc 0
stdout
stderr
Proc 1
stdout
stderr
Proc 2
stdout
stderr
PROCESSUS DISTANTS
PROCESSUS LOCAUX
stderr
stdout
stderr_pipe
stdout_pipe
dsmexec
Machine locale

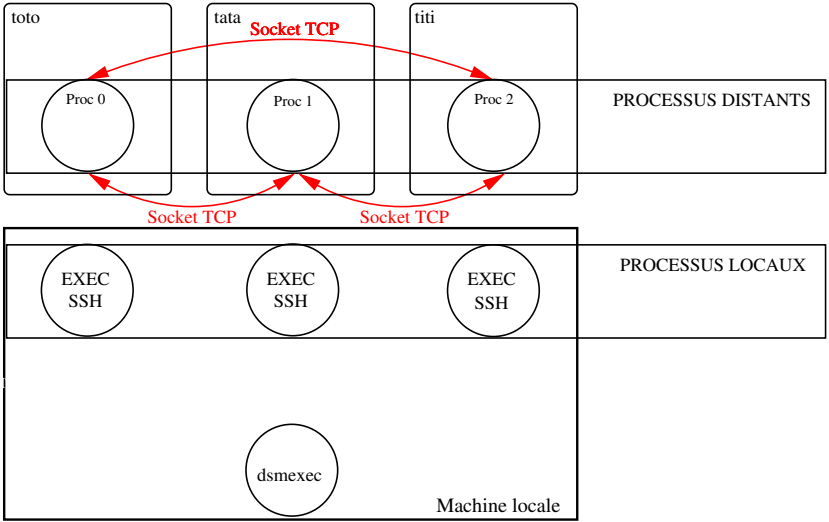

toto
tata
titi
Proc 0
Proc 1
Proc 2
stdout
stderr
PROCESSUS DISTANTS
PROCESSUS LOCAUX
stderr_pipe
stdout_pipe
dsmexec
Machine locale

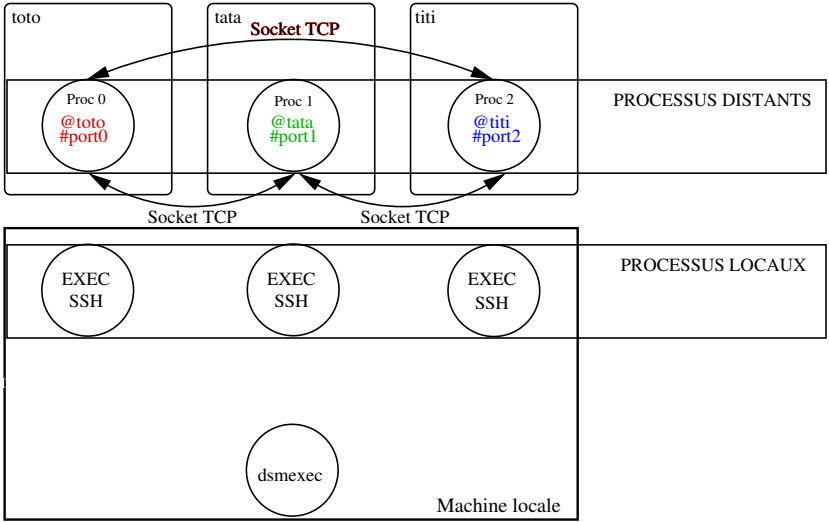

toto
tata
titi
Proc 0
Proc 1
Proc 2
stdout
stderr
PROCESSUS DISTANTS
redirection ssh (auto)
redirection ssh (auto)
redirection ssh (auto)
PROCESSUS LOCAUX
redirections tubes
redirections tubes
redirections tubes
stderr_pipe
stdout_pipe
stdout_pipe
stderr_pipe
stderr_pipe
stdout_pipe
dsmexec
Machine locale

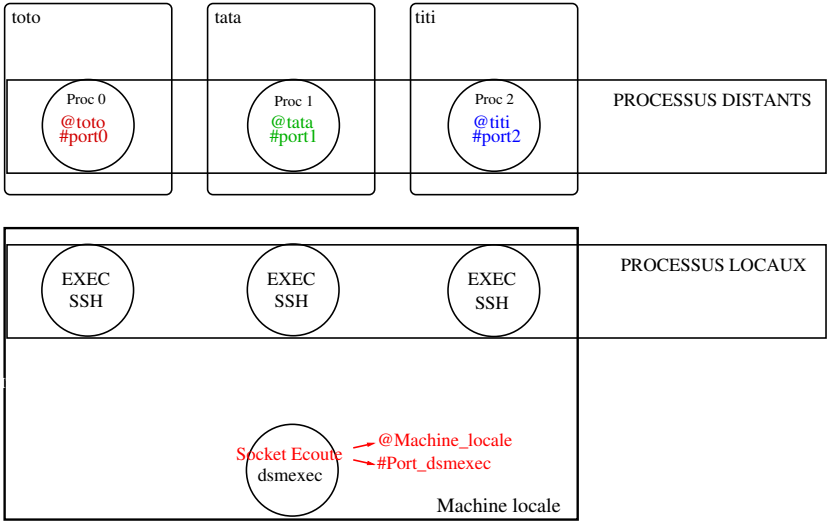

toto
tata
titi
Proc 0
stdout
stderr
Proc 1
stdout
stderr
Proc 2
stdout
stderr
PROCESSUS DISTANTS
PROCESSUS LOCAUX
dsmexec
Machine locale

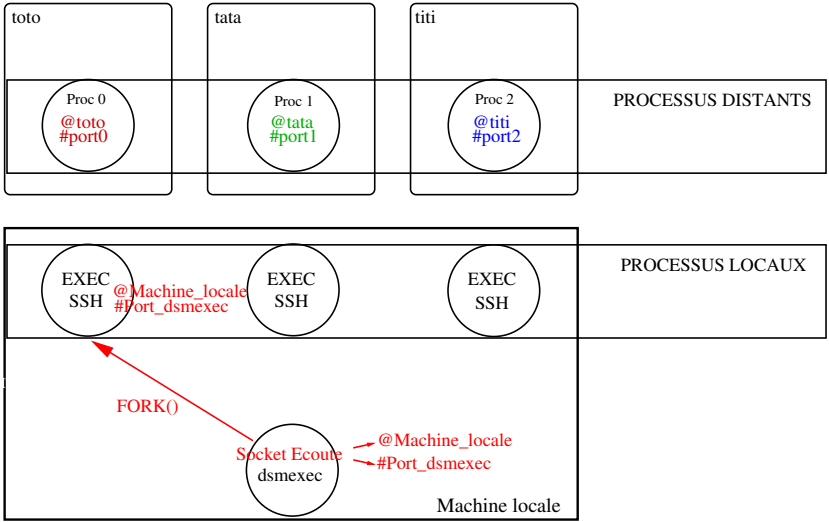

toto
tata
titi
Proc 0
stdout
stderr
Proc 1
stdout
stderr
Proc 2
stdout
stderr
PROCESSUS DISTANTS
PROCESSUS LOCAUX
lecture des tubes
dsmexec
Machine locale

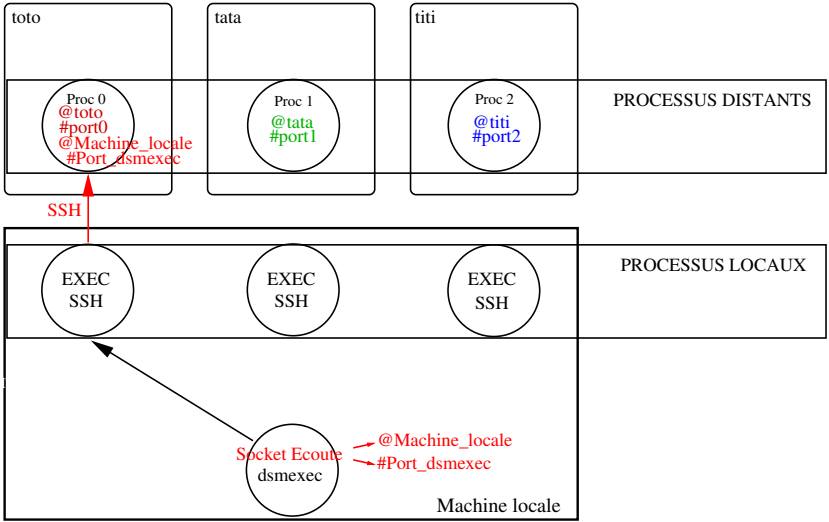

toto
tata
titi
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
EXEC SSH
EXEC SSH
EXEC SSH
PROCESSUS LOCAUX
dsmexec
Machine locale

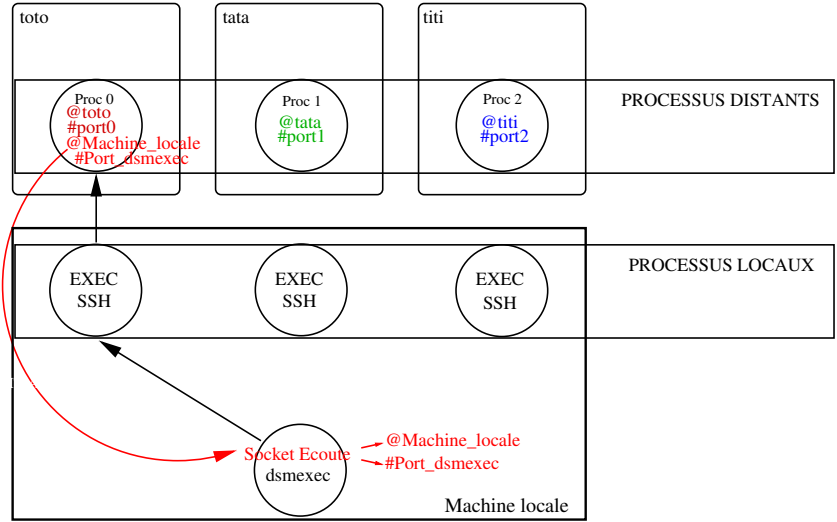

toto
tata
titi
Socket TCP
Proc 0
Proc 1
Proc 2
PROCESSUS DISTANTS
Socket TCP
Socket TCP
EXEC SSH
EXEC SSH
EXEC SSH
PROCESSUS LOCAUX
dsmexec
Machine locale

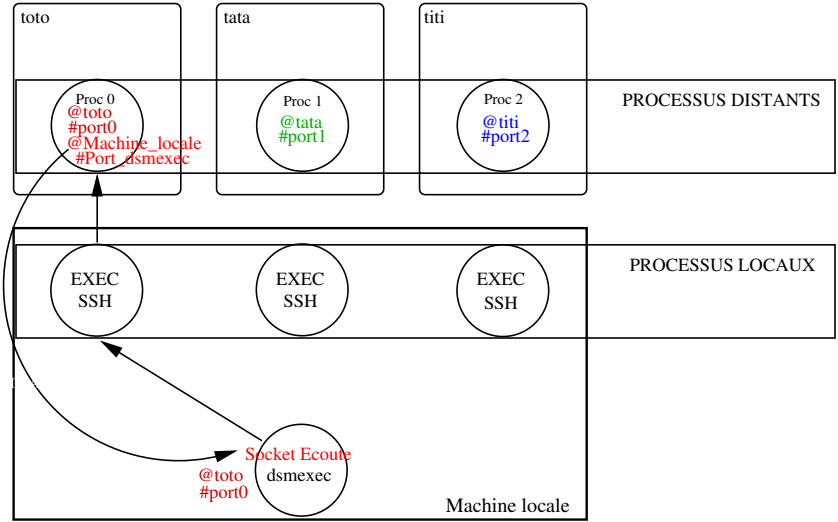

toto
tata
titi
Socket TCP
Proc 0
@toto
#port0
Proc 1
@tata
#port1
Proc 2
@titi
#port2
PROCESSUS DISTANTS
Socket TCP
Socket TCP
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
dsmexec
Machine locale

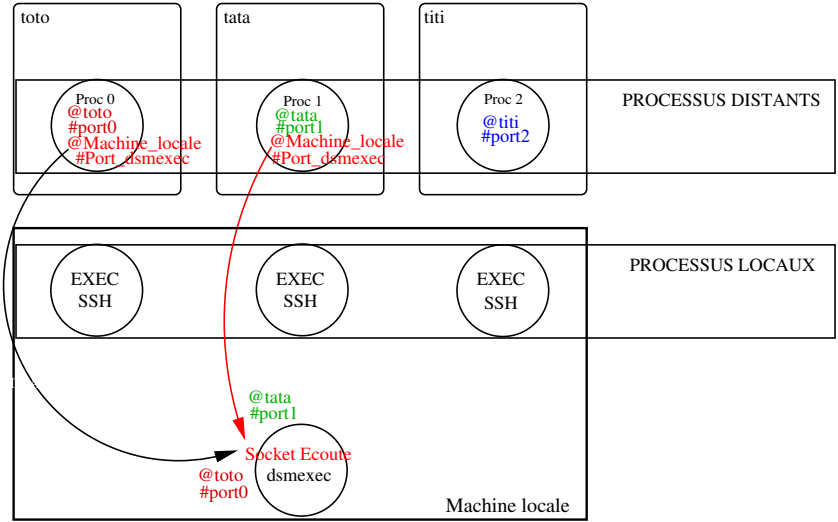

toto
tata
titi
Proc 0
@toto
#port0
Proc 1
@tata
#port1
Proc 2
@titi
#port2
PROCESSUS DISTANTS
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
Socket Ecoute
dsmexec
@Machine_locale
#Port_dsmexec
Machine locale

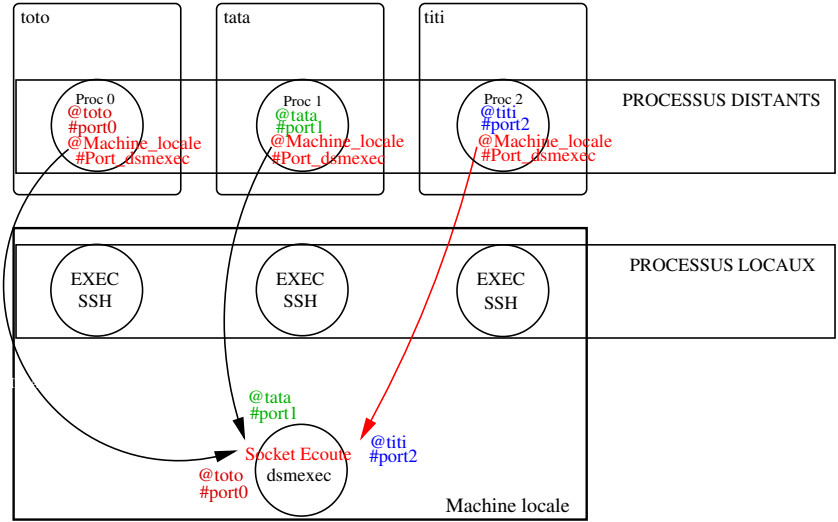

toto
tata
titi
Proc 0
@toto
#port0
Proc 1
@tata
#port1
Proc 2
@titi
#port2
PROCESSUS DISTANTS
EXEC
SSH
@Machine_locale
#Port_dsmexec
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
FORK()
Socket Ecoute
dsmexec
@Machine_locale
#Port_dsmexec
Machine locale

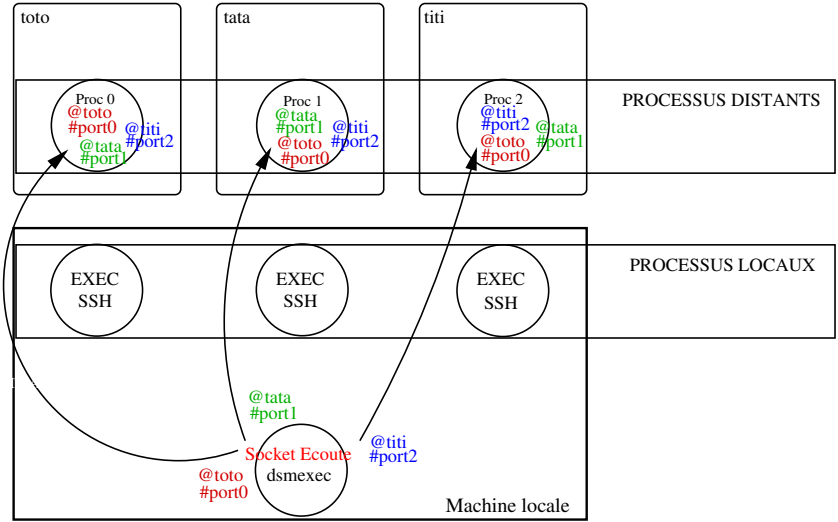

toto
tata
titi
Proc 0
@toto
#port0
@Machine_locale
#Port_dsmexec
Proc 1
@tata
#port1
Proc 2
@titi
#port2
PROCESSUS DISTANTS
SSH
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
Socket Ecoute
dsmexec
@Machine_locale
#Port_dsmexec
Machine locale

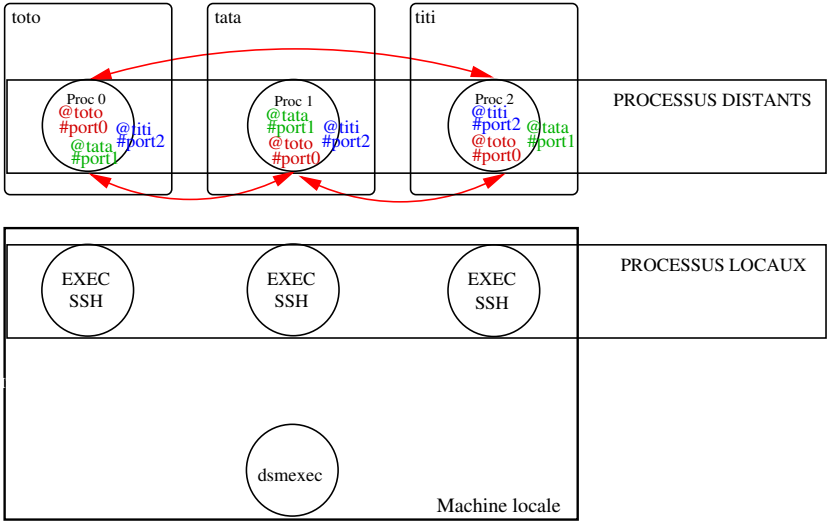

toto
tata
titi
Proc 0
@toto
#port0
@Machine_locale
#Port_dsmexec
Proc 1
@tata
#port1
Proc 2
@titi
#port2
PROCESSUS DISTANTS
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
Socket Ecoute
dsmexec
@Machine_locale
#Port_dsmexec
Machine locale


toto
tata
titi
Proc 0
@toto
#port0
@Machine_locale
#Port_dsmexec
Proc 1
@tata
#port1
Proc 2
@titi
#port2
PROCESSUS DISTANTS
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
Socket Ecoute
@toto
#port0
dsmexec
Machine locale


toto
tata
titi
Proc 0
@toto
#port0
@Machine_locale
#Port_dsmexec
Proc 1
@tata
#port1
@Machine_locale
#Port_dsmexec
Proc 2
@titi
#port2
PROCESSUS DISTANTS
EXEC SSH
EXEC SSH
EXEC SSH
PROCESSUS LOCAUX
@tata
#port1
Socket Ecoute
@toto
#port0
dsmexec
Machine locale


toto
tata
titi
Proc 0
@toto
#port0
@Machine_locale
#Port_dsmexec
Proc 1
@tata
#port1
@Machine_locale
#Port_dsmexec
Proc 2
@titi
#port2
@Machine_locale
#Port_dsmexec
PROCESSUS DISTANTS
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
@tata
#port1
Socket Ecoute
dsmexec
@titi
#port2
@toto
#port0
Machine locale


toto
tata
titi
Proc 0
@toto
#port0
@titi
#port2
@tata
#port1
Proc 1
@tata
#port1
@titi
#port2
@toto
#port0
Proc 2
@titi
#port2
@tata
#port1
@toto
#port0
PROCESSUS DISTANTS
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
@tata
#port1
Socket Ecoute
dsmexec
@titi
#port2
@toto
#port0
Machine locale


toto
tata
titi
Proc 0
@toto
#port0
@titi
#port2
@tata
#port1
Proc 1
@tata
#port1
@titi
@toto
#port2
#port0
Proc 2
@titi
#port2
@tata
@toto
#port1
#port0
PROCESSUS DISTANTS
EXEC
SSH
EXEC
SSH
EXEC
SSH
PROCESSUS LOCAUX
dsmexec
Machine locale

Phase 1 — Hypothèses sur les processus déployés

- Contexte statique : nombre de processus fixé au moment du déploiement
  - Pas de rajout de processus, pas de départ en cours d'application
  - Le nombre de processus de l'application est consultable via la variable d'environnement `DSNETEC_NBNODE`
- Tous les processus sont identifiés par un numéro unique (sans rapport avec le pid)
  - Les numéros sont contigus de 0 à `DSNETEC_NBNODE - 1`
  - Ce numéro est consultable via la variable d'environnement `DSNETEC_RANKID`

Phase 1 — Hypothèses sur les processus déployés

- Contexte **statique** : nombre de processus fixé au moment du déploiement

⇒ Pas de rajout de processus, pas de départ en cours d'application

⇒ Le nombre de processus de l'application est consultable via la **variable d'environnement** `DSMEXEC_RANKNUM`

- Tous les processus sont identifiés par un numéro unique (sans rapport avec le pid)

⇒ Les numéros sont contigus de 0 à `DSMEXEC_RANKNUM−1`

⇒ Ce numéro est consultable via la **variable d'environnement** `DSMEXEC_RANKID`

Phase 1 — Hypothèses sur les processus déployés

- Contexte **statique** : nombre de processus fixé au moment du déploiement

⇒ Pas de rajout de processus, pas de départ en cours d'application

⇒ Le nombre de processus de l'application est consultable via la **variable d'environnement** `DSMEXEC_RANKNUM`

- Tous les processus sont identifiés par un numéro unique (sans rapport avec le pid)

⇒ Les numéros sont contigus de 0 à `DSMEXEC_RANKNUM−1`

⇒ Ce numéro est consultable via la **variable d'environnement** `DSMEXEC_RANKID`

Phase 1 — Hypothèses sur les processus déployés

- Contexte **statique** : nombre de processus fixé au moment du déploiement

⇒ Pas de rajout de processus, pas de départ en cours d'application

⇒ Le nombre de processus de l'application est consultable via la **variable d'environnement** `DSMEXEC_RANKNUM`

- Tous les processus sont identifiés par un numéro unique (sans rapport avec le pid)

⇒ Les numéros sont contigus de 0 à `DSMEXEC_RANKNUM−1`

⇒ Ce numéro est consultable via la **variable d'environnement** `DSMEXEC_RANKID`

Phase 1 — Hypothèses sur les processus déployés

- Contexte **statique** : nombre de processus fixé au moment du déploiement

⇒ Pas de rajout de processus, pas de départ en cours d'application

⇒ Le nombre de processus de l'application est consultable via la **variable d'environnement** `DSMEXEC_RANKNUM`

- Tous les processus sont identifiés par un numéro unique (sans rapport avec le pid)

⇒ Les numéros sont contigus de 0 à `DSMEXEC_RANKNUM−1`

⇒ Ce numéro est consultable via la **variable d'environnement** `DSMEXEC_RANKID`

Phase 1 — Hypothèses sur les processus déployés

- Contexte **statique** : nombre de processus fixé au moment du déploiement

⇒ Pas de rajout de processus, pas de départ en cours d'application

⇒ Le nombre de processus de l'application est consultable via la **variable d'environnement** `DSMEXEC_RANKNUM`

- Tous les processus sont identifiés par un numéro unique (sans rapport avec le pid)

⇒ Les numéros sont contigus de 0 à `DSMEXEC_RANKNUM`−1

⇒ Ce numéro est consultable via la **variable d'environnement** `DSMEXEC_RANKID`

Phase 1 — Hypothèses sur les processus déployés

- Contexte **statique** : nombre de processus fixé au moment du déploiement

⇒ Pas de rajout de processus, pas de départ en cours d'application

⇒ Le nombre de processus de l'application est consultable via la **variable d'environnement** `DSMEXEC_RANKNUM`

- Tous les processus sont identifiés par un numéro unique (sans rapport avec le pid)

⇒ Les numéros sont contigus de 0 à `DSMEXEC_RANKNUM`−1

⇒ Ce numéro est consultable via la **variable d'environnement** `DSMEXEC_RANKID`

- Gestion des communications via une zone de mémoire partagée entre tous les processus
- Zone comprise entre `BASE_ADDR` et `TOP_ADDR` (valeurs identiques pour tous les processus)
- Zone mémoire organisée en pages mémoire de 4ko
- Chaque page est identifiée par un numéro
- Une page n'appartient physiquement qu'à un unique processus (le propriétaire)
- Chaque processus possède une table avec les informations concernant toutes les pages du système

- Gestion des communications via une zone de mémoire partagée entre tous les processus
- Zone comprise entre `BASE_ADDR` et `TOP_ADDR` (valeurs identiques pour tous les processus)
- Zone mémoire organisée en pages-mémoire de 4ko
- Chaque page est identifiée par un numéro
- Une page n'appartient physiquement qu'à un unique processus (le propriétaire)
- Chaque processus possède une table avec les informations concernant toutes les pages du système

- Gestion des communications via une zone de mémoire partagée entre tous les processus
- Zone comprise entre `BASE_ADDR` et `TOP_ADDR` (valeurs identiques pour tous les processus)
- Zone mémoire organisée en pages-mémoire de 4ko
- Chaque page est identifiée par un numéro
- Une page n'appartient physiquement qu'à un unique processus (le propriétaire)
- Chaque processus possède une table avec les informations concernant toutes les pages du système

- Gestion des communications via une zone de mémoire partagée entre tous les processus
- Zone comprise entre `BASE_ADDR` et `TOP_ADDR` (valeurs identiques pour tous les processus)
- Zone mémoire organisée en pages-mémoire de 4ko
- Chaque page est identifiée par un numéro
- Une page n'appartient physiquement qu'à un unique processus (le propriétaire)
- Chaque processus possède une table avec les informations concernant toutes les pages du système

- Gestion des communications via une zone de mémoire partagée entre tous les processus
- Zone comprise entre `BASE_ADDR` et `TOP_ADDR` (valeurs identiques pour tous les processus)
- Zone mémoire organisée en pages-mémoire de 4ko
- Chaque page est identifiée par un numéro
- Une page n'appartient physiquement qu'à un unique processus (le propriétaire)
- Chaque processus possède une table avec les informations concernant toutes les pages du système

- Gestion des communications via une zone de mémoire partagée entre tous les processus
- Zone comprise entre `BASE_ADDR` et `TOP_ADDR` (valeurs identiques pour tous les processus)
- Zone mémoire organisée en pages-mémoire de 4ko
- Chaque page est identifiée par un numéro
- Une page n'appartient physiquement qu'à un unique processus (le propriétaire)
- Chaque processus possède une table avec les informations concernant toutes les pages du système

- Gestion des communications via une zone de mémoire partagée entre tous les processus
- Zone comprise entre `BASE_ADDR` et `TOP_ADDR` (valeurs identiques pour tous les processus)
- Zone mémoire organisée en pages-mémoire de 4ko
- Chaque page est identifiée par un numéro
- Une page n'appartient physiquement qu'à un unique processus (le propriétaire)
- Chaque processus possède une table avec les informations concernant toutes les pages du système

Une interface minimale réduite :

- Deux fonctions
- Deux variables globales

Une interface minimale réduite :

- Deux fonctions
  - `char *dsm_init(void);`
  - `void dsm_finalize(void);`
- Deux variables globales
  - `dsm_node_id` : numéro d'un processus au sein de l'application distribuée
  - `dsm_node_num` : nombre de processus de l'application distribuée

Une interface minimale réduite :

- Deux fonctions
  1. `char *dsm_init(void);`
  2. `void dsm_finalize(void);`
- Deux variables globales
  1. `dsm_node_id` : numéro d'un processus au sein de l'application distribuée
  2. `dsm_node_num` : nombre de processus de l'application distribuée

Une interface minimale réduite :

- Deux fonctions
  1. `char *dsm_init(void);`
  2. `void dsm_finalize(void);`

- Deux variables globales
  1. `dsm_node_id` : numéro d'un processus au sein de l'application distribuée
  2. `dsm_node_num` : nombre de processus de l'application distribuée

Une interface minimale réduite :

- Deux fonctions
  1. `char *dsm_init(void);`
  2. `void dsm_finalize(void);`

- Deux variables globales
  1. `dsm_node_id` : numéro d'un processus au sein de l'application distribuée
  2. `dsm_node_num` : nombre de processus de l'application distribuée

Une interface minimale réduite :

- Deux fonctions
  1. `char *dsm_init(void);`
  2. `void dsm_finalize(void);`

- Deux variables globales
  1. `dsm_node_id` : numéro d'un processus au sein de l'application distribuée
  2. `dsm_node_num` : nombre de processus de l'application distribuée

Une interface minimale réduite :

- Deux fonctions
  1. `char *dsm_init(void);`
  2. `void dsm_finalize(void);`

- Deux variables globales
  1. `dsm_node_id` : numéro d'un processus au sein de l'application distribuée
  2. `dsm_node_num` : nombre de processus de l'application distribuée

```c
#include "dsm.h"

int main(int argc, char **argv)
{
  char *pointer = dsm_init();
  char *current = pointer;
  int   value;

  fprintf(stdout,"[Proc %i] base address for shared mapping is: %p\n", dsm_node_id, pointer);

  if(0 == dsm_node_id){
    *((int *)current) += 4;
    value = *((int *)current);
    printf("[Proc %i] integer value: %i\n", dsm_node_id, value);
  } else if(1 == dsm_node_id){
    *((int *)current) += 8;
    value = *((int *)current);
    printf("[Proc %i] integer value: %i\n", dsm_node_id, value);
  } else {
    printf("[Proc %i] I'm not part of this \n", dsm_node_id);
  }

  dsm_finalize();

  exit(EXIT_SUCCESS);
}
```

```c
#include "dsm.h"

int main(int argc, char **argv)
{
  char *pointer = dsm_init();
  char *current = pointer;
  int   value;

  fprintf(stdout,"[Proc %i] base address for shared mapping is: %p\n", dsm_node_id, pointer);

  if(0 == dsm_node_id){
    *((int *)current) += 4;
    value = *((int *)current);
    printf("[Proc %i] integer value: %i\n", dsm_node_id, value);
  } else if(1 == dsm_node_id){
    *((int *)current) += 8;
    value = *((int *)current);
    printf("[Proc %i] integer value: %i\n", dsm_node_id, value);
  } else {
    printf("[Proc %i] I'm not part of this \n", dsm_node_id);
  }

  dsm_finalize();

  exit(EXIT_SUCCESS);
}
```